Orchestration avec Airflow

Concepts fondamentaux · Conception de DAGs · Bonnes pratiques DataOps

Michel Cadennes - mars 2026

Plan du cours

1 **Pourquoi orchestrer ?**
Le problème des pipelines manuels

2 **Apache Airflow — Concepts**
DAG, Tasks, Operators, Scheduling

3 **Bonnes pratiques DataOps**
Idempotence, atomicité, versionnement

4 **Conception des DAGs SciPulse**
Batch ArXiv + Micro-batch HN

5 **Mise en pratique (TP)**
Complétion et test des DAGs

Pourquoi orchestrer ?

Le problème des pipelines manuels en production

Le problème : l'exécution manuelle

En TP, on exécute les étapes une par une :

```
python -m src.transformation.load_to_postgres   # Étape 1
cd dbt && dbt run && dbt test && cd ..          # Étape 2
python -m src.enrichment.enrich_arxiv           # Étape 3
python -m src.quality.validate_arxiv            # Étape 4
```

En production, c'est intenable.

Quatre questions sans réponse

1 Qui lance les scripts ?

Sans humain le weekend = pipeline à l'arrêt

2 Que faire si l'étape 2 échoue ?

L'étape 3 risque de s'exécuter sur des données corrompues

3 Comment relancer partiellement ?

On veut relancer l'étape 3 seule, pas tout depuis le début

4 Comment diagnostiquer ?

Sans logs centralisés, c'est un cauchemar archéologique

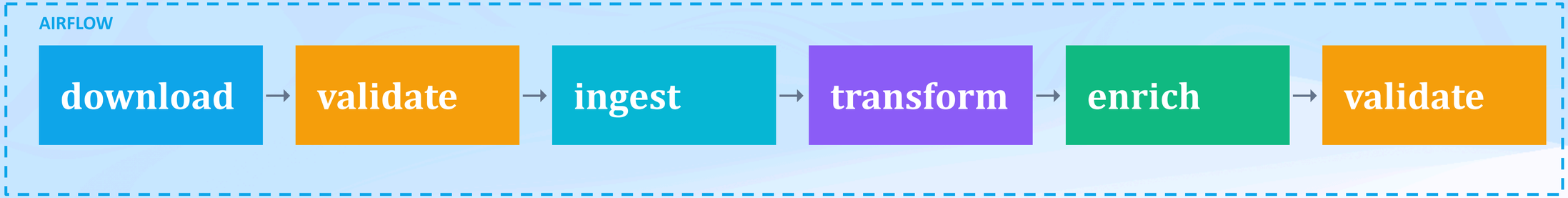
La solution : l’orchestrateur

Un orchestrateur est un système qui planifie, séquence, exécute, surveille et relance les tâches d'un pipeline automatiquement.

Il ne fait rien lui-même — il dit aux autres quoi faire.

Planifier | Séquencer | Exécuter | Surveiller | Relancer

L'orchestrateur dans l'architecture DataOps



Automatisation

Le pipeline s'exécute selon un planning, sans intervention humaine

Observabilité

Chaque exécution est loggée, les durées mesurées, les échecs déclenchent des alertes

Collaboration

Le DAG est un schéma visuel que toute l'équipe peut lire, versionné dans Git

Apache Airflow

Concepts fondamentaux de l'orchestrateur standard

Quest-ce qu’Apache Airflow

Origine

Créé par Airbnb en 2014, projet Apache depuis 2019

Rôle

Orchestrateur de workflows open-source — le standard de facto

Philosophie

Airflow déclenche les tâches, il ne les exécute pas lui-même

Définition

Garantit que les bonnes tâches s'exécutent dans le bon ordre, au bon moment

Les Graphes Orientés Acycliques (DAG)

Le concept central d'Airflow : un ensemble de tâches reliées par des dépendances.

Directed

Les flèches ont un sens :

download → validate, pas l'inverse

Acyclic

Pas de boucle : une tâche ne peut

pas dépendre d'elle-même

Graph

Branchements (parallélisme)

et convergences (synchronisation)

Pipeline == Code

Défini en Python

- Un DAG est un fichier Python
- Pas un fichier YAML, pas une interface graphique
- Versionné dans Git
- Revu en pull request
- Testé automatiquement

```
download
  >> validate_raw
  >> ingest
  >> transform
  >> enrich
  >> validate_final
```

Tâches et opérateurs

Une Task est une unité de travail. Chaque tâche utilise un Operator pour encapsuler une action.

Operator	Ce qu'il fait	Exemple SciPulse
BashOperator	Exécute une commande shell	dbt run, curl, scripts shell
PythonOperator	Exécute une fonction Python	Enrichissement NLP, validation GX
EmptyOperator	Point de synchronisation	Jonction entre branches parallèles

Ces trois opérateurs couvrent 90% des cas. L'opérateur déclenche le travail — le travail est dans le script externe.

La programmation (scheduling)

Chaque DAG a un schedule_interval qui définit sa fréquence d'exécution.

Expression	Fréquence
`None`	Déclenchement manuel uniquement
`"@daily"`	Tous les jours à minuit
`"@hourly"`	Toutes les heures
`"*/5 * * * *"`	Toutes les 5 minutes (cron)
`"0 6 * * 1"`	Tous les lundis à 6h

Dans SciPulse

ArXiv pipeline
schedule = None

(manuel / quotidien)

HN Poller
schedule = "*/5 * * * *"

(toutes les 5 minutes)

Retry, gestion des échecs

Circuit Breaker DataOps

Si une tâche échoue après tous ses retries :

→ Le DAG s'arrête

→ Les tâches en aval ne s'exécutent pas

→ Les données corrompues ne se propagent pas

Valeurs par défaut :

"retries": 3

"retry_delay": 5 min

"retry_exponential_backoff": True

"email_on_failure": True

"execution_timeout": 2 hours

Communication entre tâches

Tâche A — Push

```
xcom_push(
    key='doc_count',
    value=100000
)
```

→

Tâche B — Pull

```
xcom_pull(
    task_ids=« task_a",
    key="doc_count"
)
```

Limites des XComs

Stockés dans la base PostgreSQL d'Airflow. Conçus pour de petites valeurs (nombres, chaînes, listes courtes). Pour les données volumineuses, passer par un stockage partagé : MinIO, Elasticsearch, PostgreSQL.

Architecture d’Airflow

Scheduler
Le cerveau d'Airflow

Lit les fichiers DAG, détermine les tâches à exécuter, les soumet à l'executor

Executor
Les bras d'Airflow

Exécute les tâches. Types : LocalExecutor, CeleryExecutor, KubernetesExecutor

Webserver
localhost:8080

Interface web pour visualiser les DAGs, suivre les exécutions, consulter les logs

Metadata DB
PostgreSQL séparé

Stocke l'état de chaque DAG, tâche et exécution. La mémoire d'Airflow

Bonnes pratiques DataOps

Principes pour des DAGs fiables et maintenables

DAG versions dans git

Versionnement

- Les fichiers DAG sont du code Python
- Versionnés dans le dépôt Git du projet
- Revus en pull request
- Testés avant déploiement

Idempotence

- Chaque tâche doit être re-exécutable sans effet de bord.
- C'est ce qui permet de relancer une tâche après un échec sans corrompre les données.

Exemples SciPulse : document_id dans ES, DROP+CREATE dans PG, upsert dans le poller HN

Tâches atomiques et focalisées

Mauvais

ingest_and_transform

Si cette tâche échoue, on ne sait pas si c'est l'ingestion ou la transformation.

Correct

ingest >> transform

Deux tâches séparées = diagnostic immédiat en cas d'échec.

depends_on_past: False

Par défaut, Airflow peut conditionner l'exécution d'une tâche à la réussite de la même tâche dans le run précédent.

En DataOps, c'est indésirable : si le run de lundi a échoué, on veut que celui de mardi puisse s'exécuter (les données de mardi ne dépendent pas de celles de lundi).

Sensors : déclenchement événementiel

Un Sensor est un type de tâche qui attend qu'une condition soit remplie avant de continuer.

```
FileSensor(
    task_id="wait_for_arxiv",
    filepath="/data/*.json",
    poke_interval=60,
    timeout=3600
)
```

Cas d'usage

- Fichier dans MinIO
- API disponible
- Autre DAG terminé

Pattern « event-driven orchestration » — plus élégant qu'un cron fixe quand le timing est imprévisible.

Conception des DAG SciPulse

Deux DAG pour deux rythmes

scipulse_arxiv_pipeline

Pattern : Batch

Fréquence : Manuel / quotidien

Tâches : 6 tâches séquentielles

Source : ArXiv (dump quotidien)

scipulse_hn_poller

Pattern : Micro-batch

Fréquence : Toutes les 5 minutes

Tâches : 3 tâches séquentielles

Source : Hacker News (flux continu)

Pourquoi pas un seul DAG ?

Les deux pipelines sont indépendants en timing. Les coupler forcerait le poller HN à attendre la fin du batch ArXiv — une perte de réactivité.

Le DAG pour ArXiv

1	download_arxiv_dump	Téléchargement
2	validate_raw_arxiv	Checkpoint GE (données brutes)
3	ingest_arxiv_logstash	Déclencher Logstash
4	transform_dbt	dbt run + dbt test
5	enrich_arxiv_python	TF-IDF + croisement HN
6	validate_enriched	Checkpoint GE (données enrichies)

Chaque flèche est une dépendance : si une tâche échoue, les suivantes ne s'exécutent pas.

Le DAG Hacker News

poll_hn_api
Interroge l'API Hacker News

detect_arxiv_links
Détecte les liens vers ArXiv

update_arxiv_hn_scores
Met à jour les scores HN

Configuration clé

Schedule	Toutes les 5 minutes
max_active_runs	1 (un seul run actif)
Sécurité	Si un cycle > 5 min, le suivant attend
Pattern	Quasi-temps réel

Mise en pratique

Ce qui vient dans le TP

1	Ajout d'Airflow	Vérification dans docker-compose.yml (déjà dans le starter kit)
2	Complétion DAG ArXiv	Remplacer les TODO par les vrais appels aux scripts des Jours 1-3
3	Complétion DAG HN	Connecter le poller et la détection de liens
4	Test des DAGs	Trigger manuel, visualisation UI, inspection des logs
5	Simulation d'échec	Erreur volontaire pour observer retry et arrêt du DAG

Bibliographie

[1]	**Harenslak & de Ruiter (2021)**	*Data Pipelines with Apache Airflow — Manning*
[2]	**Apache Airflow (2024)**	*Documentation officielle — airflow.apache.org*
[3]	**Reis & Housley (2022)**	*Fundamentals of Data Engineering — O'Reilly*
[4]	**Kleppmann (2017)**	*Designing Data-Intensive Applications — O'Reilly*
[5]	**Apache Airflow (2024)**	*Operators Reference*
[6]	**Astronomer (2024)**	*Airflow Best Practices — astronomer.io*
[7]	**DataOps Manifesto**	*dataopsmanifesto.org*
[8]	**Apache Airflow (2024)**	*XComs Documentation*
[9]	**Beauchemin (2015)**	*The Rise of the Data Engineer*